Desarrollo de Aplicación Híbridas

UT1: Introducción y Entorno de Desarrollo

UT1: Introducción y Entorno de Desarrollo

¡Bienvenido/a a la primera Unidad de Trabajo! Este es el punto de partida de nuestro viaje para convertirnos en desarrolladores de aplicaciones híbridas. Aquí sentaremos las bases teóricas y, lo más importante, prepararemos nuestro ordenador con todas las herramientas necesarias para empezar a construir.

Objetivos de la Unidad

Al finalizar esta unidad, serás capaz de:

- Diferenciar entre una aplicación nativa, una web-app y una aplicación híbrida.
- Entender el papel que juegan tecnologías como Ionic y Capacitor en el desarrollo móvil.
- Instalar y configurar un entorno de desarrollo profesional en tu propio equipo.
- Crear y ejecutar tu primera aplicación híbrida básica.

¡Vamos a empezar!

Obra publicada con **Licencia Creative Commons Reconocimiento Compartir igual 4.0 <http://creativecommons.org/licenses/by-sa/4.0/>**

Desarrollo de Aplicación Híbridas

1.1. Conceptos Clave: El Ecosistema Híbrido

1.1. Conceptos Clave: El Ecosistema Híbrido

Tecnologías Web: La Base de Todo

Las aplicaciones que construiremos utilizan las mismas tecnologías que ya conoces del desarrollo web:

- **HTML:** Para estructurar el contenido de la aplicación.
- **CSS:** Para darle estilo y diseño visual.
- **JavaScript:** Para añadir interactividad y lógica de programación.

La gran ventaja es que no necesitas aprender un nuevo lenguaje de programación para crear aplicaciones móviles.

Frameworks UI: Ionic

Un framework como **Ionic** nos proporciona una librería de componentes de interfaz de usuario (UI) pre-diseñados y optimizados para móviles. En lugar de crear un botón o una tarjeta desde cero, Ionic nos da componentes como `<ion-button>` o `<ion-card>` que se ven y se sienten como componentes nativos tanto en iOS como en Android.

Puentes Nativos: Capacitor

Aquí está la magia. ¿Cómo accede nuestra app web a la cámara, al GPS o a los contactos del teléfono? A través de un "puente" nativo como **Capacitor**. Capacitor empaqueta nuestra aplicación web dentro de un contenedor nativo y nos proporciona una API en JavaScript para comunicarnos con las funcionalidades del dispositivo. Cuando llamas a la función para abrir la cámara desde tu código, Capacitor "traduce" esa orden al lenguaje nativo del móvil.

Ventajas e Inconvenientes del Desarrollo Híbrido

Ventajas	Inconvenientes
Un solo código base: Escribes el código una vez y funciona en iOS, Android y la Web.	**Rendimiento:** Aunque ha mejorado mucho, para tareas muy intensivas (juegos 3D, edición de vídeo) puede no ser tan fluido como el nativo.
Desarrollo más rápido y económico: Menos tiempo y recursos al no tener que mantener dos equipos de desarrollo (uno para iOS, otro para Android).	**Dependencia de plugins:** Para acceder a las últimas funcionalidades nativas, a veces hay que esperar a que la comunidad o el equipo de Capacitor actualice el plugin correspondiente.
Fácil de aprender: Si ya sabes desarrollo web, la curva de aprendizaje es mucho menor.	**Limitaciones de diseño:** Aunque es muy personalizable, conseguir una apariencia 100% idéntica a una app nativa específica puede requerir trabajo extra.

Obra publicada con **Licencia Creative Commons Reconocimiento Compartir igual 4.0 <http://creativecommons.org/licenses/by-sa/4.0/>**

Desarrollo de Aplicación Híbridas

1.2. Práctica Guiada: Instalación del Entorno

1.2. Práctica Guiada: Instalación del Entorno

Vamos a preparar nuestro ordenador para convertirlo en una estación de desarrollo de aplicaciones. Sigue estos pasos con atención.

1. Editor de Código: Visual Studio Code

Es el editor más popular para desarrollo web y móvil. Si no lo tienes, descárgalo e instálalo desde su web oficial.

Enlace: Descargar Visual Studio Code <https://code.visualstudio.com/>

2. Gestor de Versiones de Node: nvm (Opcional pero recomendado)

Diferentes proyectos pueden requerir diferentes versiones de Node.js. `nvm` (Node Version Manager) te permite instalar y cambiar entre versiones fácilmente. Busca "nvm-windows" si usas Windows o "nvm" para macOS/Linux y sigue sus instrucciones de instalación.

3. Entorno de Ejecución: Node.js y npm

Node.js es el entorno que nos permite ejecutar JavaScript fuera del navegador, y `npm` es su gestor de paquetes, que usaremos para instalar Ionic y otras herramientas.

1. Si instalaste `nvm`, abre una nueva terminal y escribe:

   ```
   nvm install --lts

   nvm use --lts
   ```

2. Si no instalaste `nvm`, ve a la web oficial de Node.js y descarga la versión **LTS** (Long Term Support).
3. **Verificación:** Abre una terminal (o reiníciala si la tenías abierta) y comprueba que se han instalado correctamente con estos comandos. Deberían devolverte un número de

versión:

```
node -v
```

```
npm -v
```

4. Interfaz de Línea de Comandos (CLI) de Ionic

Esta es la herramienta principal que usaremos para crear, construir y ejecutar nuestros proyectos de Ionic.

1. Abre tu terminal.
2. Escribe el siguiente comando para instalar la CLI de Ionic de forma global en tu sistema:

   ```
   npm install -g @ionic/cli
   ```

3. **Verificación:** Una vez termine la instalación, ejecuta:

   ```
   ionic -v
   ```

¡Listo! Si todos los comandos de verificación te han devuelto una versión, tu entorno de desarrollo está preparado.

Obra publicada con **Licencia Creative Commons Reconocimiento Compartir igual 4.0 <http://creativecommons.org/licenses/by-sa/4.0/>**

Desarrollo de Aplicación Híbridas

1.3. Reto: Tu Primera App "Hola Mundo"

1.3. Reto: Tu Primera App "Hola Mundo"

Ahora que tienes todo el entorno configurado, es el momento de crear tu primera aplicación con Ionic. Este primer proyecto nos servirá para comprobar que todo funciona correctamente y para familiarizarnos con la estructura básica de una app.

Paso 1: Crear el proyecto con `ionic start`

> #### ¡Importante! Tarea de Documentación
>
> A medida que realizas este reto y los siguientes de la Unidad 1, ve haciendo **capturas de pantalla** de los pasos clave: la terminal después de crear un proyecto, la aplicación funcionando en el navegador, el código que has modificado en VS Code, etc. ¡Las necesitarás para el manual de instalación que deberás entregar al final de la unidad!

Abre una terminal o línea de comandos, navega a la carpeta donde guardas tus proyectos (por ejemplo, `Documentos/ProyectosDAH`) y ejecuta el siguiente comando:

```
ionic start mi-primera-app tabs
```

> #### ¿Y si no sé qué plantilla usar?
>
> Si ejecutas el comando `ionic start` sin especificar el nombre de la plantilla al final, la CLI de Ionic iniciará un asistente interactivo y te mostrará una lista con todas las

> plantillas disponibles (`tabs`, `sidemenu`, `blank`, etc.) para que puedas elegir la que prefieras. ¡Pruébalo!

Al ejecutar el comando, la CLI de Ionic te hará algunas preguntas:

- **Framework a utilizar:** Asegúrate de seleccionar **Angular**.
- **Tipo de proyecto (si aparece la pregunta):** Si la CLI te da a elegir entre `NgModules` o `Standalone`, selecciona siempre **Standalone**. En este curso usaremos este enfoque moderno.

Como ya le hemos dado el nombre ("mi-primera-app") y la plantilla ("tabs") en el propio comando, no nos lo volverá a preguntar. El proceso de creación tardará unos minutos, ya que descargará todas las dependencias necesarias.

Paso 2: Explorar las Plantillas (Templates)

Ionic nos ofrece varias plantillas para empezar. El último parámetro del comando `ionic start` (`tabs` en nuestro caso) define la estructura inicial. Te animo a que crees otros dos proyectos para ver las diferencias:

```
# Un proyecto con un menú lateral desplegable
ionic start mi-app-sidemenu sidemenu

# Un proyecto con una página en blanco, ideal para empezar desde cero
ionic start mi-app-blank blank
```

Explora las carpetas de los tres proyectos y fíjate en cómo cambia la estructura de las páginas en `src/app/`.

Paso 3: Lanzar la aplicación en el navegador

Una vez creado un proyecto, el siguiente paso es verlo en acción. Para ello:

1. Navega con la terminal a la carpeta de uno de tus nuevos proyectos (ej: `cd mi-primera-app`).
2. Ejecuta el siguiente comando:

```
ionic serve
```

Este comando compilará la aplicación y la abrirá automáticamente en tu navegador web. ¡Verás tu primera app de Ionic funcionando!

Paso 4: Haz tu primer cambio

Para terminar, vamos a hacer una pequeña modificación para que veas lo fácil que es. Si abres el archivo `src/app/tab1/tab1.page.html`, verás que la etiqueta `<ion-title>` aparece dos veces. ¿Por qué?

> ### El Efecto de Cabecera "Condensada" (Específico de iOS)
>
> Ionic utiliza esta doble cabecera para crear un efecto visual muy común en las apps nativas, pero que por defecto **solo se activa en el modo iOS**. Funciona así:
>
> - El `<ion-title size="large">` (el título grande) es el que se ve al entrar en la página cuando la app está en modo iOS.
> - El `<ion-title>` normal (el título pequeño) es el que se muestra siempre en el modo Android (MD), y también es al que se encoge el título grande en modo iOS cuando el usuario hace scroll.
>
> Aunque el efecto solo se vea en iOS, es una buena práctica cambiar el texto en ambos sitios para mantener la coherencia.

1. Abre la carpeta del proyecto `mi-primera-app` en Visual Studio Code.
2. Busca el archivo `src/app/tab1/tab1.page.html`.
3. Localiza las **dos** etiquetas `<ion-title>` que contienen el texto "Tab 1".
4. Cambia el texto en ambas por "Mi Primera App".
5. Guarda el archivo y comprueba que el título ha cambiado en el navegador.

Paso 5: Añadir contenido y probar el scroll

Tal y como está ahora, la página tiene muy poco contenido y no podemos hacer scroll para ver el efecto de la cabecera. Vamos a añadir texto de relleno para solucionarlo.

1. En el mismo archivo `tab1.page.html`, busca el `<div id="container">`.
2. Dentro de ese `div`, debajo de la línea que contiene el enlace, pega varias veces un párrafo. Puedes copiar y pegar el siguiente bloque de código:

```
<!-- Pega este bloque dentro del div id="container" -->
<p>Este es un párrafo de relleno para hacer la página más larga.</p>
```

```
    <p>Este es un párrafo de relleno para hacer la página más larga.</p>
    <p>Este es un párrafo de relleno para hacer la página más larga.</p>
    <p>Este es un párrafo de relleno para hacer la página más larga.</p>
    <p>Este es un párrafo de relleno para hacer la página más larga.</p>
```

Ahora, si vuelves a tu navegador, verás que la página es más larga y ya puedes hacer scroll.

> ### ¡No veo el título grande! ¿Por qué?
>
> Como hemos explicado, el efecto de la cabecera condensada es un patrón de diseño del **modo iOS**. Por defecto, tu navegador muestra la app en **modo MD (Material Design / Android)**, donde este efecto no está activo.
>
> Para verlo funcionar, solo tienes que añadir un parámetro a la URL en tu navegador: `?ionic:mode=ios`
>
> Por ejemplo: `http://localhost:8100/?ionic:mode=ios`
>
> Al recargar la página con esa URL, verás el estilo de iOS y el título grande aparecerá. ¡Prueba a hacer scroll para ver el efecto en acción!

Obra publicada con **Licencia Creative Commons Reconocimiento Compartir igual 4.0 <http://creativecommons.org/licenses/by-sa/4.0/>**

Desarrollo de Aplicación Híbridas

1.4. Reto Avanzado: Ejecución en Emulador y Dispositivo Físico

1.4. Reto Avanzado: Ejecución en Emulador y Dispositivo Físico

En el reto anterior, lanzamos nuestra aplicación en el navegador. Ahora daremos el salto al mundo nativo. En este reto, aprenderás a instalar y configurar las herramientas necesarias para ejecutar tu proyecto Ionic como una verdadera aplicación en un emulador de Android y en tu propio teléfono.

Este proceso es más complejo que el anterior, ¡así que presta mucha atención a cada paso!

```
# ¡Importante!
# No olvides hacer capturas de pantalla de los pasos clave.
# Las necesitarás para la tarea final de documentación.
```

Parte 1: Instalar Android Studio

Android Studio es el entorno de desarrollo oficial para Android. Aunque no escribiremos código directamente en él, lo necesitamos porque instala y gestiona el **SDK de Android** (el kit de herramientas para crear apps de Android) y los emuladores.

1. Ve a la página oficial de Android Studio <https://developer.android.com/studio> y descarga la última versión para tu sistema operativo.
2. Sigue el asistente de instalación con las opciones por defecto. Es una instalación larga, así que ten paciencia.
3. Una vez instalado, abre Android Studio. La primera vez que se ejecute, probablemente te pida instalar componentes adicionales del SDK. Acepta todas las instalaciones recomendadas.

Requisitos Previos Importantes: JDK y Variables de Entorno

Para que la comunicación entre Ionic y el SDK de Android funcione correctamente, hay dos configuraciones críticas que debes verificar.

1. Instalar el JDK (Java Development Kit) versión 21

Capacitor (la herramienta que usa Ionic) requiere una versión específica del JDK para construir aplicaciones de Android. A día de hoy, la recomendada es la **versión 21**.

- Puedes descargar OpenJDK 21 desde la página oficial de Java <https://jdk.java.net/21/> .
- Descarga el archivo ZIP o el instalador para tu sistema operativo y sigue las instrucciones.
- Asegúrate de que el JDK esté correctamente instalado ejecutando `java -version` en tu terminal. Deberías ver una salida que mencione la versión 21.

2. Configurar la variable de entorno ANDROID_HOME

Necesitas decirle a tu sistema operativo dónde está instalado el SDK de Android. Esto se hace creando una variable de entorno llamada `ANDROID_HOME`.

- **En Windows:**
  1. Abre el menú de inicio y busca "Editar las variables de entorno del sistema".
  2. En la ventana de "Propiedades del sistema", haz clic en "Variables de entorno...".
  3. En la sección "Variables de usuario", haz clic en "Nueva...".
  4. **Nombre de la variable:** `ANDROID_HOME`
  5. **Valor de la variable:** La ruta a tu SDK de Android. Por defecto suele ser: `C:\Users\TU_USUARIO\AppData\Local\Android\Sdk` (recuerda cambiar "TU_USUARIO").
  6. Acepta todas las ventanas. Es posible que necesites reiniciar la terminal (o el ordenador) para que los cambios surtan efecto.
- **En macOS o Linux:**
  1. Abre una terminal.
  2. Edita el archivo de configuración de tu shell (normalmente `~/.zshrc` en macOS modernos, o `~/.bash_profile` o `~/.bashrc` en Linux y macOS antiguos). Puedes usar el editor nano: `nano ~/.zshrc`.
  3. Añade la siguiente línea al final del archivo. La ruta por defecto suele ser: `export ANDROID_HOME=$HOME/Library/Android/sdk`
  4. Guarda el archivo (`Ctrl+O` en nano) y sal (`Ctrl+X`).
  5. Recarga la configuración de tu terminal ejecutando `source ~/.zshrc` o simplemente abriendo una nueva ventana de terminal.

Parte 2: Crear un Emulador de Android

Un emulador es un dispositivo virtual que se ejecuta en tu ordenador. Es perfecto para hacer pruebas rápidas.

1. En la pantalla de bienvenida de Android Studio, haz clic en "More Actions" -> "Virtual Device Manager".
2. Haz clic en "Create device".
3. Elige un modelo de teléfono (por ejemplo, "Pixel 7") y haz clic en "Next".
4. Se te mostrará una lista de imágenes del sistema Android. Elige la última versión estable (sin la etiqueta "Tiramisu" o "Upside Down Cake" si aparecen, busca una con un nombre de API numérico alto) y haz clic en el icono de descarga que aparece a su lado.
5. Una vez descargada, selecciónala y haz clic en "Next".
6. Puedes dejar las opciones avanzadas como están. Haz clic en "Finish".
7. ¡Listo! Ahora deberías ver tu nuevo emulador en la lista. Puedes cerrar el Virtual Device Manager.

Parte 3: Lanzar la App en el Emulador y Dispositivo Físico

Ahora viene la magia. Con las herramientas de Ionic, podemos compilar y lanzar nuestra aplicación en Android con un solo comando.

Paso 1: Compilar y Ejecutar

Navega con tu terminal a la carpeta de tu proyecto Ionic y ejecuta el siguiente comando. Este es el comando principal que usarás a partir de ahora para probar en Android:

```
ionic capacitor run android
```

Consejo: También puedes usar la abreviatura `ionic cap run android` para ejecutar el mismo comando.

Al ejecutarlo, Ionic hará varias cosas automáticamente:

- **Si es la primera vez**, añadirá la plataforma nativa de Android a tu proyecto (equivalente a `ionic capacitor add android`).
- Revisará si hay cambios y construirá la versión web de tu app (como con `ionic build`).
- Sincronizará esos cambios con el proyecto nativo de Android (como con `npx cap sync`).
- Te mostrará en la terminal una lista para que elijas en qué dispositivo quieres lanzar la app.

Paso 2: Elegir el dispositivo

En la terminal, verás una lista de "dispositivos disponibles".

- **Para ejecutar en el emulador:** Simplemente selecciona el emulador que creaste en la Parte 2 y pulsa "Enter". El emulador se iniciará (si no lo estaba ya) y tu aplicación se instalará y abrirá en él.
- **Para ejecutar en un dispositivo físico:**
    1. En tu teléfono Android, ve a "Ajustes" -> "Acerca del teléfono" y pulsa repetidamente sobre "Número de compilación" hasta que aparezca un mensaje que diga "¡Ahora eres desarrollador!".
    2. Vuelve a "Ajustes" -> "Sistema" y busca "Opciones para desarrolladores".
    3. Activa la opción "Depuración por USB".
    4. Conecta tu teléfono al ordenador con un cable USB. **¡Aviso!** Asegúrate de que es un **cable de datos** y no uno de "solo carga". Muchos cables baratos solo sirven para cargar la batería pero no para transferir datos. Si tu teléfono no aparece en la lista, prueba con otro cable.
    5. En el teléfono, aparecerá una ventana pidiendo permiso para la depuración. Acepta.
    6. Ahora, tu teléfono físico debería aparecer en la lista de dispositivos disponibles. Selecciónalo y pulsa "Enter".

¡Felicidades! Has ejecutado tu primera aplicación Ionic en un entorno nativo. A partir de ahora, cada vez que quieras probar tus cambios, solo tienes que volver a ejecutar `ionic cap run android`.

Resolución de Problemas: ¿Qué hacer si algo falla?

El proceso de compilación nativa tiene muchas piezas móviles y es normal que algo falle. Aquí tienes una guía de los problemas más comunes y cómo solucionarlos.

1. Mi dispositivo no aparece en la lista

Para diagnosticar por qué tu emulador o teléfono no es detectado, puedes usar una herramienta de Capacitor llamada `native-run`.

Abre una terminal y ejecuta el siguiente comando:

```
npx native-run android --list --verbose
```

Este comando le pide a tu sistema que liste todos los dispositivos Android que tiene detectados y que te dé una respuesta muy detallada. Si hay un problema con el SDK de Android, con las variables de entorno o con la conexión ADB de tu teléfono, este comando te dará una pista muy clara sobre el error.

2. Pasa un "Chequeo Médico" con Capacitor Doctor

Ionic incluye una herramienta de diagnóstico aún más potente que comprueba toda tu configuración de Android (JDK, variables de entorno, SDK, etc.). Es lo primero que deberías ejecutar si tienes problemas.

```
npx cap doctor android
```

El "doctor" analizará tu entorno y te dirá exactamente qué está mal y, en muchos casos, cómo solucionarlo.

3. Errores relacionados con "Gradle"

Gradle es el sistema que Android usa para construir el proyecto. A veces, la primera compilación puede fallar si la descarga de Gradle se interrumpe. Si ves errores que mencionan "Gradle", puedes intentar limpiar el proyecto nativo:

- **En Windows:** Abre una terminal y ejecuta `cd android && gradlew clean && cd ..`
- **En macOS/Linux:** Abre una terminal y ejecuta `cd android && ./gradlew clean && cd ..`

Tras ejecutar este comando, vuelve a intentar `ionic cap run android`. Esto forzará a Gradle a reconstruir el proyecto desde cero.

```
# ¡Importante!
# No olvides hacer capturas de pantalla de los pasos clave.
# Las necesitarás para la tarea final de documentación.
```

Obra publicada con **Licencia Creative Commons Reconocimiento Compartir igual 4.0 <http://creativecommons.org/licenses/by-sa/4.0/>**

Desarrollo de Aplicación Híbridas

1.5. Tarea Final UT1: Manual de Instalación del Entorno

1.5. Tarea Final UT1: Manual de Instalación del Entorno

Para finalizar esta primera unidad y evaluar el criterio de documentación, tu tarea consiste en crear un **manual de instalación completo y detallado**. Debes imaginar que este manual será utilizado por un nuevo compañero o compañera que no tiene ningún conocimiento previo sobre el desarrollo de aplicaciones híbridas.

Objetivo de la Tarea

Crear un documento técnico, claro y bien estructurado que guíe a cualquier persona a través del proceso completo de configuración del entorno de desarrollo de Ionic en su ordenador.

Contenido Obligatorio del Manual

Tu documento debe incluir, como mínimo, las siguientes secciones:

1. **Portada:** Con el título del manual, tu nombre y la fecha.
2. **Índice:** Una tabla de contenidos que permita navegar por el documento.
3. **Introducción:** Una breve explicación del propósito del manual y una descripción de cada una de las herramientas que se van a instalar (VS Code, Node.js, Ionic CLI, Android Studio).
4. **Secciones de Instalación Detalladas:** Una sección para cada herramienta con:
    - Explicación paso a paso del proceso de instalación.
    - **Capturas de pantalla de cada paso crucial.** Esto es muy importante. Debes incluir capturas de la descarga, del asistente de instalación, de la terminal, etc.
    - Capturas de pantalla que demuestren la correcta instalación (ej: el resultado de los comandos `node -v`, `npm -v`, `ionic -v`).
    - Captura de pantalla del Administrador de Dispositivos Virtuales (AVD) de Android Studio mostrando el emulador que has creado.
5. **Sección de Troubleshooting (Solución de Problemas):** Describe al menos un problema común que te hayas encontrado o que imagines que podría ocurrir (ej: el comando

'ionic' no se reconoce, un error de permisos en npm, etc.) y explica cómo lo solucionarías.

Formato y Entrega

- **Formato:** El manual debe entregarse en formato **PDF**.
- **Herramientas:** Puedes usar cualquier procesador de texto (Microsoft Word, Google Docs, LibreOffice Writer) para crearlo y luego exportarlo a PDF.
- **Entrega:** El archivo PDF se subirá a la tarea correspondiente en la plataforma Moodle.

Esta tarea es fundamental para demostrar que no solo has configurado tu entorno, sino que has comprendido el proceso y eres capaz de comunicarlo de manera técnica y profesional.

Obra publicada con **Licencia Creative Commons Reconocimiento Compartir igual 4.0 <http://creativecommons.org/licenses/by-sa/4.0/>**

Desarrollo de Aplicación Híbridas

1.6. Recursos Adicionales

1.6. Recursos Adicionales

La documentación oficial es la mejor amiga de un desarrollador. Aquí tienes los enlaces a las tecnologías que usaremos. ¡Te recomiendo guardarlos en tus marcadores!

- **Documentación de Ionic Framework:** https://ionicframework.com/docs <https://ionicframework.com/docs>
- **Documentación de Angular:** https://angular.io/docs <https://angular.io/docs>
- **Documentación de Capacitor:** https://capacitorjs.com/docs <https://capacitorjs.com/docs>
- **Página oficial de Node.js:** https://nodejs.org/ <https://nodejs.org/>

Obra publicada con **Licencia Creative Commons Reconocimiento Compartir igual 4.0** <http://creativecommons.org/licenses/by-sa/4.0/>